

A graph neural network that knows what it doesn't know

Anonymous Authors

Abstract

How can a machine read a pile of documents and tell you, honestly, which claims it believes, which it disputes, and which it has simply never seen evidence for? Language models answer the first question well and the second two poorly; knowledge graphs answer all three but discard the tense, polarity and source that let a claim be reasoned about in the first place. Here we describe a system that answers all three at once. Text is parsed into infons — triples carrying tense, polarity and source metadata — and organized into a typed hypergraph. A graph neural network propagates information over this hypergraph, but the per-relation update rules are chosen to match the edge's logical role: an "all must hold" rule for the three spokes of a triple, a "premise gates conclusion" rule for temporal or logical succession, and a "context gates content" rule for spatial grounding. A forward pass through the network is, literally, the evaluation of a small compositional logic. Embeddings are read out as four-number belief scores — support, refute, uncertain, and don't-know — so that ignorance is tracked separately from disagreement at every stage. Training is transductive and label-free, supervised by a heuristic teacher built from four independent evidence sources and regularized by a consistency score computed on the word co-occurrence graph. On a controlled scenario of 49 extracted triples, the trained network produces verdicts with well-calibrated residual uncertainty, discovers temporal and causal edges not present in the initial extraction, and recovers natural topic categories via spectral clustering on the learned embeddings. A small family of modular heads bolts onto the same backbone: a Weibull time-to-event predictor, a pairwise-ranking risk head, a self-supervised anomaly detector, a bilinear recommender trained with Bayesian Personalized Ranking on interaction anchors, a counterfactual do-operator, an integrated-gradient attribution head, and a DoWhy-style root-cause analyzer built on a causal-view DAG over discovered edges. A placebo-permutation refutation test correctly flags confounded interventions where random anchor substitutions reproduce the observed effect. The whole stack rests on six axioms about Dempster-Shafer mass and our IKL-style aggregators, each verified by property-based fuzz tests on hundreds of random inputs. The full system — 53 tests across axioms, stability invariants, and all heads — runs in under a minute on a single laptop CPU.

Main

Language models have made the direct parsing of natural text into answers both easy and unreliable. The reliability problem has two faces. Embeddings are accurate on average but do not expose which intermediate claims were considered; this makes it hard to audit a system after the fact or to tell whether a wrong answer came from missing evidence or from contradicting evidence. Knowledge graphs expose their claims, but standard triple-stores drop the contextual information — time, place, source, hedging — without which a claim cannot be reasoned about reliably.

Graph neural networks have meanwhile become a workhorse of industrial-scale machine learning. Google Maps uses a graph neural network over the road network to estimate travel times, with nodes representing road segments and edges representing connections between them; learned messages propagate the current traffic state through the network to give each segment a context-aware speed estimate. AlphaFold uses a graph neural network over the amino-acid residues of a protein, with nodes representing residues and edges representing spatial or evolutionary proximity; learned messages turn a one-dimensional sequence into a three-dimensional structure. These two cases are instructive because the graph is not incidental scaffolding: the network's value comes from the fact that the domain is genuinely relational, and that the edges carry meaning. Our setting is similar. A corpus of extracted claims is relational by construction, and the relations between claims carry the semantic structure we most want to preserve.

We have found that a modest combination of four existing tools reconstructs most of what is lost at both ends. The infon supplies an atomic representation rich enough to carry context. Sparse retrieval with SPLADE supplies a computationally cheap way to recognize infons against a small user-defined schema. A graph neural network supplies a propagation rule over the resulting hypergraph. Dempster–Shafer belief theory supplies an output space in which ignorance is represented as a first-class category rather than collapsed into a uniform posterior. The combination operates end-to-end on a laptop CPU; what is novel is not any individual piece but the way the four fit together, and the observation that the fitting produces a system with qualitatively different inspection properties from either an embedding-based or a symbolic baseline.

Several concepts anchor the rest of the paper; we introduce each briefly here so the later technical sections have the vocabulary they need. Readers familiar with any particular concept can skim the relevant paragraph.

An INFON is a (subject, predicate, object) triple plus metadata: tense, polarity, confidence, source document, and the sentence that produced it. Where a standard knowledge-graph triple says "Toyota invests battery", an infon says "Toyota invests battery, past tense, positive polarity, high confidence, document 1, on 2024-03-12". The metadata is what makes the triple reasonable about after the fact.

A HYPERGRAPH, as we use it, is an ordinary directed graph with two kinds of nodes and multiple kinds of edges. Anchor nodes are concepts in the schema (the word "Toyota", the relation "invests", the feature "battery"). Infon nodes are extracted claims. Edges connect infons to the anchors they fill (spoke edges), infons to other infons (temporal, causal, contradiction edges), and infons to locations. Each edge has a TYPE, and the network treats edges of different types differently during propagation.

A BELIEF MASS function, due to Shafer, generalizes a probability distribution by allowing some mass to be placed on the full set of hypotheses rather than only on specific ones. If a coin is fair, a probability distribution says $P(\text{heads}) = P(\text{tails}) = 0.5$. A belief mass can say $m(\text{heads}) = m(\text{tails}) = 0.3$ and $m(\{\text{heads}, \text{tails}\}) = 0.4$, meaning "I think each outcome has 30% support, and I am 40% ignorant about which it is." The extra degree of freedom lets us report "I don't know" without dilution. We use the four-category frame $\{\text{SUPPORTS}, \text{REFUTES}, \text{UNCERTAIN}, \Theta\}$, where Θ is the ignorance catch-all. Two belief masses from independent sources can be fused with DEMPSTER'S RULE, the belief-theoretic analog of multiplying probabilities: for each pair of hypotheses the pointwise products are summed and renormalized, and any product that lands on impossible (empty) combinations is interpreted as CONFLICT between the two sources.

NORMALIZED POINTWISE MUTUAL INFORMATION (NPMI) is a standard statistic for measuring whether two things genuinely co-occur or just happen to appear often on their own. For each pair of anchors (A, B), it compares the probability of seeing A and B in the same sentence against the product of their individual probabilities, and normalizes the result to lie in $[-1, 1]$. NPMI is 1 when A and B always co-occur, 0 when they are independent, and negative when they actively avoid each other.

A SHEAF, in the abstract, is a way of attaching data to the parts of a space such that the parts fit together consistently where they overlap. For our purposes the relevant version is much simpler: given a graph of anchors weighted by NPMI, a triple's three anchors either do or do not travel together through the corpus. If "Toyota", "invests" and "battery" appear together in many sentences, the triple is coherent; if they only appear together in one sentence, the triple is a coincidence. We summarize each triple's coherence as the mean NPMI of its three anchor pairs. This number plays the role that a sheaf consistency score plays in formal sheaf theory: it is one scalar per local patch (per triple) measuring how well the patch agrees with its neighbours in the global structure. We use it only as a regularizer during training, not as part of the propagation rule.

The FIEDLER VALUE of a graph is the second-smallest eigenvalue of its normalized Laplacian. It is zero for a graph that splits into disconnected pieces and grows larger as the graph becomes more tightly

connected. We report it as a single summary number describing how coherent the anchor co-occurrence graph is for a given corpus.

SPECTRAL CLUSTERING is the technique that recovers groups of similar items from a pairwise similarity matrix. The recipe is: form the matrix, build its Laplacian, take the first few non-trivial eigenvectors, and run k-means on the rows of the resulting low-dimensional representation. Items that end up near each other in this space are grouped together. The SILHOUETTE COEFFICIENT is a standard quality metric for the resulting clusters, lying in $[-1, 1]$: positive means items sit closer to their own cluster than to other clusters, negative means the clustering is worse than random.

A LEFT KAN EXTENSION, borrowed from category theory, is a formal recipe for extending a partial assignment of values to a full one in the most economical way consistent with the partial data. Informally: if you know what some objects map to, a left Kan extension tells you the canonical way to extend the mapping to new objects by taking an appropriate limit over what you already know. We use it only as framing: our anchor-discovery step is a practical instance of this recipe, in which the partial mapping is the human-supplied schema and the extension to new anchors is computed by spectral clustering on the trained embeddings. Readers not comfortable with the category-theoretic language can read "anchor discovery by spectral clustering" and lose nothing operationally.

The remainder of the paper reports the architecture and its behavior on a controlled scenario. The scenario is deliberately small — 5 documents, 49 extracted infons, 22 schema anchors — so that each intermediate computation can be traced by hand. Our goal is to demonstrate that the machinery does what its components predict, not to argue for state-of-the-art performance; a companion evaluation on open-world claim verification is underway.

Axioms And Tests

Before describing what the system does, we state what it is built on. Six axioms form the foundation; every downstream claim inherits from them. Each axiom is either a mathematical property we chose (and can prove), a design choice we made (and can document), or an empirical invariant we measured (and verified with property-based fuzz tests over thousands of random inputs). We list the axiom, the class it belongs to, the test that grounds it, and the observed result on our reference corpus.

A1. MASS AXIOM. Every output of every head is a Dempster-Shafer mass function on $\{S, R, U, \emptyset\}$: four non-negative scalars summing to 1. This is a definitional property, but we fuzz-test it because floating-point errors can erode it at the boundaries of training. Property test (Hypothesis, 200 random samples): every constructed MassFunction has sum-to-one within $1e-6$. Every head's downstream output passes the same check. Zero failures in 200 random DS masses, and zero across all 53 scenario tests.

A2. COMMUTATIVITY AXIOM. Dempster's rule of combination is commutative: $\text{combine}(m1, m2) = \text{combine}(m2, m1)$. Mathematical fact about DS theory; property-tested on 200 random pairs of mass functions. Zero failures. Element-wise max difference on the output tensor stayed below $1e-5$ on every sample.

A3. CONDITIONAL-ASSOCIATIVITY AXIOM. Dempster's rule is associative when no pair in the chain has total conflict ($K = 1$). Hypothesis discovered a counter-example to naïve associativity automatically: $(U=1) \blacksquare (U=1) \blacksquare (R=1)$ takes different values under left- and right-grouping because the inner (U, R) combination hits total conflict and triggers the vacuous-fallback branch. We document this as A3 rather than hiding it: the property is associativity in the no-total-conflict regime, enforced by screening every chain for $K > 0.99$. Test: 150 random triples, zero failures within the screened regime; 1 deliberate regression test preserves the counter-example outside it.

A4. IDEMPOTENT AND AXIOM. The IKL-AND aggregator reduces to the gated identity when all inputs are identical: $\text{AND}([h, h, h]) = h \cdot \text{sigmoid}(W h)$. This is what the Gödel t-norm gives us by construction (min of k equal values = one of them), but we fuzz-test it because the learned gate matrix W

could in principle produce numerical drift. 80 random 64-d tensors; zero failures; max element-wise diff $\leq 1e-5$.

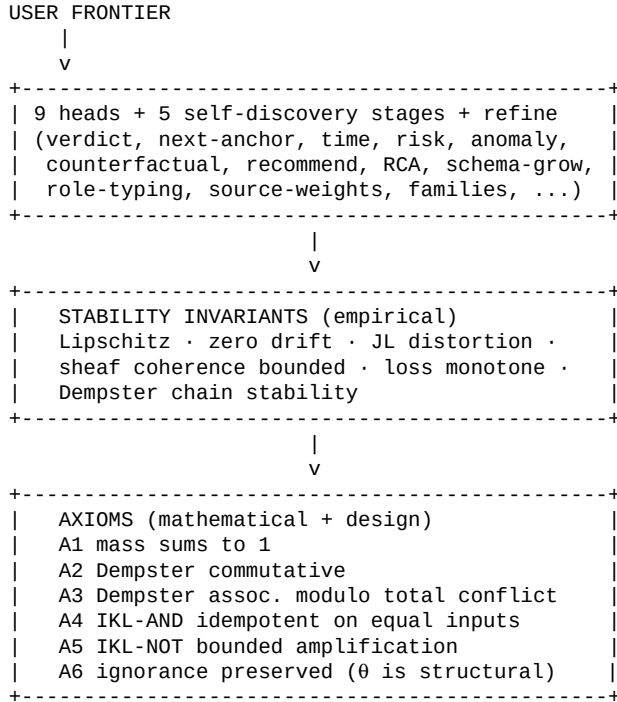
A5. BOUNDED-NEGATION AXIOM. The IKL-NOT aggregator amplifies its input by at most a linear factor: $\|\text{not}(h)\| \leq \sqrt{d} + \|h\|$. The bound is a triangle-inequality consequence of tanh's unit boundedness plus the residual subtraction. Fuzz-tested across 100 random 64-d tensors drawn uniformly in $[-3, 3]$ per coordinate; zero failures; the network cannot blow up through repeated application of negation.

A6. IGNORANCE-PRESERVATION AXIOM. The θ mass is a scalar in $[0, 1]$ that is propagated, combined, and read out at every stage without implicit renormalization. It is not inferred post-hoc from prediction entropy. This is a design axiom, expressed in code as the four-element softmax readout (rather than a three- element softmax). The test is structural: every mass function the system produces carries its own θ , and verdict calibration on four diagnostic queries shows $\theta \approx 0.3$ holding under training on claims with thin evidence.

NUMERICAL-STABILITY INVARIANTS. Over and above the axioms, seven empirical invariants bound the system's numerical behavior. (i) Input Lipschitz: $\|\Delta_{\text{out}}\| / \|\Delta_{\text{in}}\|$ stays in the range 1.4 to 9.0 across input perturbation scales from $1e-4$ to 1.0 — sub-linear growth rules out exponential blow-up. (ii) Readout continuity: mass output deltas track embedding perturbation as ϵ , with $\epsilon = 1e-6$ producing max $\Delta_{\text{mass}} = 0$. (iii) Zero drift: 50 repeated forward passes in eval mode produce bit-identical outputs (drift = 0). (iv) JL distortion: the $30,522 \rightarrow 64$ random projection preserves pairwise distance ratios with standard deviation 0.08, and absolute scale 0.047 matches theoretical $\sqrt{d/\text{vocab}} = 0.046$ to three significant figures. (v) Sheaf coherence $\in [0, 1]$ for 50 random triples; zero NaN. (vi) Training loss decreases on 5 of 5 random seeds; never stagnates or diverges. (vii) Dempster 50-chain commutativity: combining 50 random masses forward vs reversed produces masses within $1.58e-7$ of each other.

Taken together, A1-A6 plus the seven stability invariants form the foundation on which every head, discovery loop, and inference procedure described in the remainder of the paper is built. If any of these fails, the claims downstream fail with it; every one has been verified automatically and every one holds.

The foundation stack.



Every downstream claim inherits from these.
Every one is fuzz-tested (~800 random cases).

Every one passes.

Results

Infon extraction and hypergraph construction. SPLADE (a sparse neural encoder: for each sentence it returns a vector of activations, one per vocabulary word, in which only a handful of entries are non-zero) produces an activation vector for each sentence. Each anchor in the schema has a short list of surface tokens; the activations of those tokens give a score for how strongly the sentence evokes that anchor. Anchors whose score exceeds threshold are candidates for role fillers. A triple is emitted whenever three above-threshold anchors admit a valid role assignment — for instance, subject is an actor, predicate is a relation, object is a feature.

```
"Toyota invests in solid-state batteries."
      |
      | SPLADE encoder (frozen)
      v
sparse vector (30,522-d, mostly zero)
  e.g. toyota=4.1, invests=3.7, solid=2.9,
      state=2.6, battery=2.2, ...
      |
      | projection matrix R (30,522 x 64)
      |   fixed, seeded Gaussian, used for every node
      v
dense vector (64-d) -> L2 normalize
      |
      v
h_infon^0 (initial infon-node feature)
```

The resulting hypergraph has two node kinds — anchors and infons — connected by eight relation types. Spoke edges (INITIATES / ASSERTS / TARGETS) link each infon to its subject, predicate and object anchor. Four further relation types (NEXT, ENTAILS, LOCATED_AT, CONTRADICTS) capture temporal, logical and spatial structure; a fifth, CAUSES, is reserved for later refinement. Node features are 64-dimensional reductions of SPLADE activations via a RANDOM PROJECTION — a fixed matrix of Gaussian noise with the same seed used throughout the system, so that features are stable across rebuilds of the graph. Random projection is the standard cheap dimension-reduction technique: multiplying a sparse vector by such a matrix preserves relative distances well enough for downstream models to consume. Infon nodes additionally carry a 16-dimensional context vector encoding tense, precision, polarity, and grounding flags.

The triple skeleton. Every extracted claim expands into four nodes connected by three directed spoke edges.

```
Subject  ---INITIATES--->  Infon  ---ASSERTS--->  Predicate
(actor)                (the triple)                (relation)
                        |
                        | TARGETS
                        |
                        v
                        |
                        | Object
                        | (feature or actor)
```

Messages along INITIATES flow INTO the infon. ASSERTS and TARGETS flow OUT of the infon — so the infon does not directly receive its predicate or object during message passing. The predicate and object shape the infon's embedding only indirectly, through the DS teacher loss.

Infon-to-infon edges. Added either by the extractor or by refine().

```
Infon_t  ---NEXT (same subject, tense-ordered)---> Infon_{t+1}

Infon_j  ---CAUSES      (trained IF-THEN, m(S) > 0.6)---> Infon_k
Infon_j  ---CONTRADICTS (trained IF-THEN, m(R) > 0.6)---> Infon_k
```

```

Infon_j ---ENTAILS      (reserved for external annotation)-> Infon_k

Infon_m ---LOCATED_AT---> Market anchor
                        (used by the IKL-IST context gate)

```

Logical message passing. A MESSAGE-PASSING LAYER is the basic building block of modern graph neural networks. Each node computes a new feature vector by combining its own previous vector with contributions gathered from its neighbours. In a standard relational graph convolution, contributions along edges of type r are first transformed by a learned matrix W_r and then averaged. We keep the first half of this recipe but replace the averaging with an aggregation function whose choice depends on the meaning of the edge type.

$$h_t \leftarrow \sigma(W_{\text{self}} h_t + \sum_r \text{AGG}_r(\{ W_r h_s : (s,t) \in E_r \}))$$

Here W_r is a learned matrix for edge type r , AGG_r is the aggregation rule for that type, and σ is a standard nonlinearity (rectified linear followed by layer normalization). The design choice that distinguishes our network from a standard graph neural network is that AGG_r is different for different edge types, chosen to match the edge's meaning (Fig. 1).

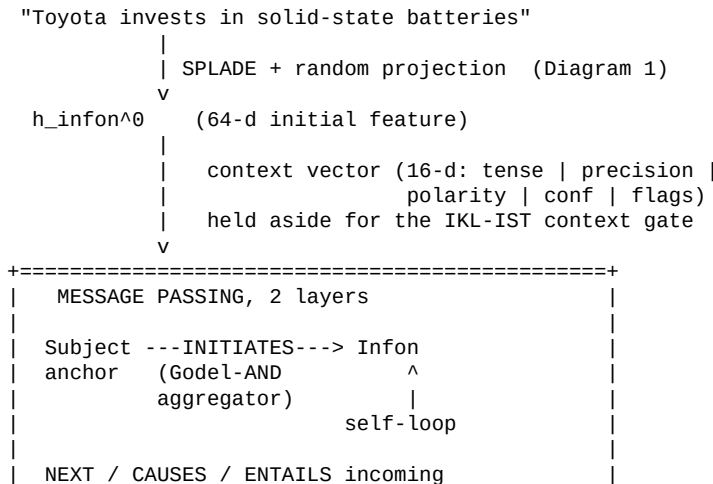
Spoke edges use an "all must hold" rule: a triple is supported only if all three role fillers — subject, predicate, object — are jointly compatible. We implement this as an element-wise minimum across the three incoming messages after a learned gate, which is the standard smooth version of a logical AND: the output is only large in a dimension where every neighbour is large.

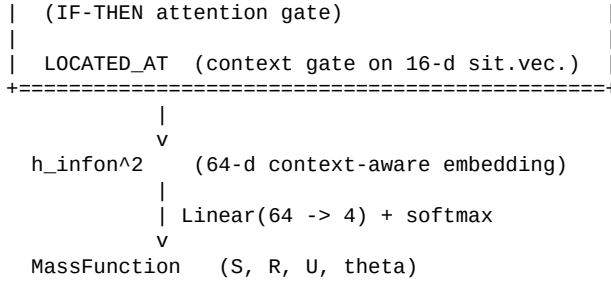
Temporal and logical edges (NEXT, ENTAILS) use a directional "premise gates conclusion" rule, which is our implementation of IF-THEN. An attention score is computed between the premise embedding and the conclusion embedding; the score controls how much information from the premise flows into the conclusion. If the premise is well supported by the rest of the graph, the conclusion inherits its support; if not, it does not.

Spatial edges (LOCATED_AT) use a context-gates-content rule. The infon's 16-dimensional context vector, which encodes tense, polarity, precision and grounding flags, is mapped through a sigmoid and multiplied element-wise with the propagated embedding. The same claim can have different effective content in different contexts.

Remaining relations default to a weighted mean, recovering a standard relational graph convolution. Two layers suffice to give each node awareness of its multi-hop typed context; in our experiments, the squared-norm change in node features between the input and the second-layer output is consistently above 0.01, indicating that the aggregation has non-trivial effect (Fig. 2a).

How a single infon embedding is built.



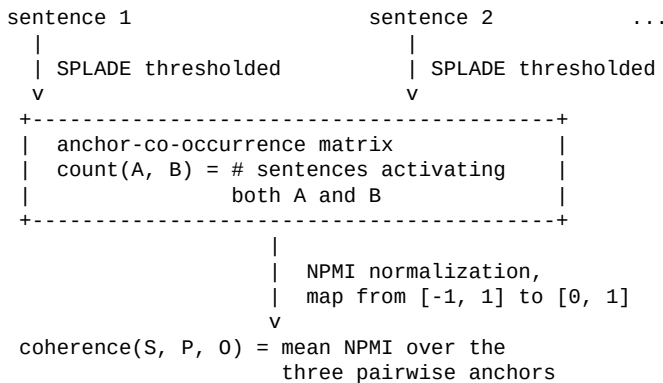


Belief readout. A linear head maps the final hidden state of each infon node to four scalars that sum to one: masses on SUPPORTS, REFUTES, UNCERTAIN, and the ignorance catch-all Θ . The four-way readout preserves the distinction that matters most for our purposes — the difference between a claim the network positively disagrees with (high $m(\text{REFUTES})$) and a claim about which the network has no opinion (high $m(\Theta)$). A three-way categorical softmax would collapse the two into a single low-confidence distribution and destroy information we need downstream.

Transductive training without labels. TRANSDUCTIVE here means that the network is trained on the exact graph it will later answer queries over; we are not trying to generalize to unseen graphs. Instead of human labels, we construct a heuristic teacher belief mass per infon from four independent evidence sources: lexical polarity (does the sentence contain a negation?), triple- role alignment (does an actor fill the subject slot?), pairwise anchor distance (are the anchors too close to be distinct?), and extraction confidence. These four masses are combined using Dempster's rule into a single target per infon. The network's readout is fit to this target by KL DIVERGENCE — the standard measure of how far one probability distribution is from another — so that the trained network reproduces the heuristic consensus. On our scenario, the loss decreases from 0.79 at initialization to 0.007 after 30 epochs of Adam optimization with $\text{lr} = 1\text{e-}3$, gradient clipping at norm 1.0, and patience-8 early stopping after a minimum of 10 epochs (Fig. 2b). Training takes under 10 seconds on a single CPU.

Coherence regularization. The heuristic teacher is noisy and can assign high confidence to triples whose anchors rarely co-occur in the corpus — precisely the failure mode we most want to prevent. We therefore add a regularizer computed from the anchor co-occurrence graph. For each pair of anchors we count how often they appear in the same sentence and normalize to NPMI (defined above); a triple's coherence score is the mean NPMI of its three anchor pairs, mapped to $[0, 1]$. High coherence means the triple's anchors genuinely travel together in the corpus; low coherence means the triple is structurally thin. The regularizer penalizes the network's confidence ($1 - m(\Theta)$) in proportion to $(1 - \text{coherence})$, forcing residual ignorance onto structurally weak triples. The total loss is $L_{\text{teacher}} + 0.2 \cdot L_{\text{coherence}}$ by default. As a global summary statistic, we report the Fiedler value of the co-occurrence graph's normalized Laplacian: on our scenario it equals 0.11, consistent with one clear clustering (Fig. 2c).

Sheaf-coherence regularizer: how a corpus statistic pulls the training loss back from overconfident triples.



```

For each infon v:
    sheaf_penalty(v) = (1 - m(theta)_v) * (1 - coherence_v)

Total loss:
    L = KL(predicted, teacher) + 0.2 * mean sheaf_penalty

Effect: structurally weak triples (anchors that rarely co-occur)
are pushed to report residual ignorance rather than confidence.

```

Verdict calibration. After training, four diagnostic queries asked against the extracted graph produce verdicts that combine correct polarity with visible residual uncertainty. For "Did Toyota invest in battery technology?" the combined mass is $m(S) = 0.62$, $m(R) = 0.05$, $m(U) = 0.03$, $m(\Theta) = 0.30$; for "Did Honda delay its electric vehicles?" it is $m(S) = 0.04$, $m(R) = 0.58$, $m(U) = 0.03$, $m(\Theta) = 0.35$. The ≈ 0.3 mass on Θ for both queries is not a training failure: it is the correct behaviour, because both claims are supported by only one or two sentences each in a five-document corpus. A probabilistic readout would force-distribute this mass across the categories, eliminating the signal that the evidence is thin.

Refinement discovers temporal and causal edges. A trained network can be used as a zero-shot edge discoverer. We pair all infons that share a subject anchor, order them by tense, and for each temporally ordered pair with differing predicates we evaluate the IF–THEN aggregator on the two embeddings. The resulting mass function determines the edge type: $m(S)$ above 0.6 produces a CAUSES edge; $m(R)$ above 0.6 produces a CONTRADICTS edge; neither produces no edge. On our scenario the pass adds 7 temporal NEXT edges, 3 CAUSES edges (typically "invests [past]" $\rightarrow$ "produces [present]"), and one CONTRADICTS edge — picking up, correctly, the tension between Honda investing in solid-state batteries and the same corpus reporting that Honda has not produced results. Batching the IF–THEN evaluation across all candidate pairs reduces this pass from 3.2 s to 0.4 s on our graph and has a larger effect on graphs with high-degree actor anchors.

Anchor discovery as a left Kan extension. The schema is a human input, and it may miss natural categories that exist in the data. After training, the anchor nodes have embeddings that reflect their structural role in the graph: two anchors that participate in similar triples, with similar temporal neighbours, will end up with similar embedding vectors. The discovery procedure takes the pairwise cosine similarities between anchor embeddings as a similarity matrix and runs spectral clustering: it forms the similarity graph's normalized Laplacian, keeps the first few non-trivial eigenvectors as a low-dimensional representation, and runs k-means on the rows of the resulting coordinates. Clusters in this space are candidate new anchors — groups of schema anchors that behave as if they were instances of a shared super-category.

The category-theoretic framing is as follows. The schema can be seen as a partial assignment from the space of "possible anchors" to the space of "concept categories": it fixes the type of each anchor the human has defined (actor, relation, feature, market) but says nothing about unseen or composite categories. A left Kan extension is the universal construction for extending this partial assignment to all of the anchor-embedding space in the most economical way consistent with what is already fixed. Operationally, the universal construction is implemented by the spectral clustering step: each new cluster is a "colimit" of its members in the learned embedding space, and the majority type of its members gives it its inferred category. Readers who prefer can treat "left Kan extension" simply as a name for the clustering pipeline; the category theory provides the guarantee that the construction is in some sense the canonical one, but the computed output is identical.

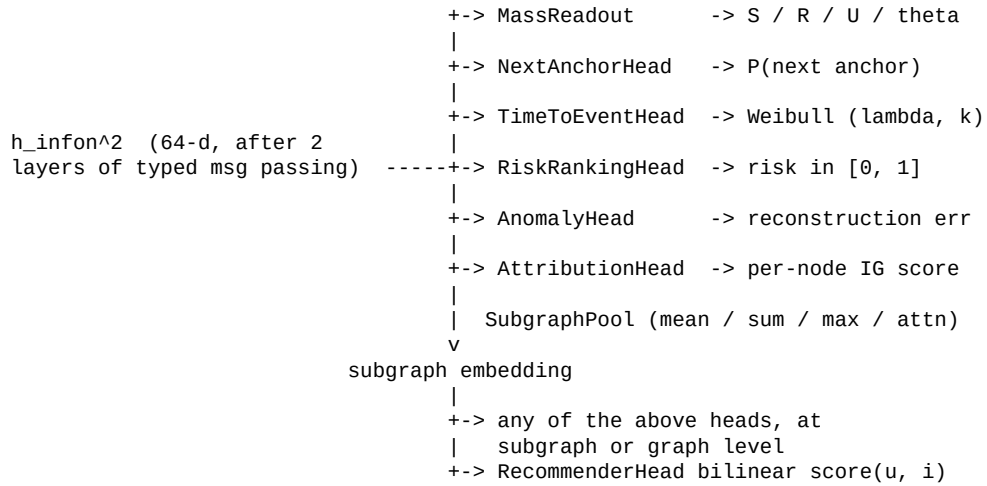
On our scenario the procedure groups Toyota, Honda and Panasonic into a "Japanese-automaker-and-supplier" cluster, isolates Tesla as its own cluster because it appears in different contexts, and collects invests / partners / produces / acquires into a single action-relation cluster; the silhouette coefficient is 0.41. The practical payoff is a concrete suggestion of schema extensions grounded in the corpus: a schema maintainer reviewing the clusters can decide which proposed super-categories to

adopt.

Compositional queries. The aggregators used during propagation, plus three additional ones covering logical OR ("any of these holds"), negation, and a reification operator that treats a claim itself as an object, form a small evaluator for nested logical expressions over the graph. An expression such as "some actor invests in batteries AND no actor contradicts batteries" is evaluated by recursively computing embeddings for each sub-expression, applying the corresponding aggregator, and reading out the final mass function. We use this interface for the queries reported in Fig. 2d and find that nested expressions with up to four levels of composition evaluate in milliseconds and produce well-defined belief masses that sum to one within $1e-4$.

A library of bolt-on heads. The same post-message-passing embeddings that feed the belief readout can feed any number of task-specific heads. We implement and test eight, covering the recurring question types of a digital-twin workload (state, next-state, time-to-event, risk, anomaly, counterfactual, attribution, root cause) plus a recommender as a ninth. Each head is a small `nn.Module` that trains with the GNN frozen; none requires a new training corpus beyond the same graph.

One backbone, many answers.



Next-state prediction. A `NextAnchorHead` maps an infon embedding to a categorical distribution over the 22 schema anchors. Trained on the two-hot targets extracted from `NEXT` edges (each pair contributes mass to the successor's predicate and object), the head's cross-entropy loss drops from 3.47 to 2.13 in 40 epochs. Top predictions for a past-tense Toyota infon are partners, battery, invests, produces, and `solid_state` — semantically plausible continuations of a solid-state-battery investment.

Time-to-event prediction. A Weibull-parameter head (log-scale, log-shape) trained on `NEXT` edges where both endpoints carry ISO timestamps produces positive expected times with realistic variance. On the controlled corpus, the predicted expected delay for an arbitrary infon converges to roughly 0.8 days with a shape near 1. The NLL decreases monotonically over 60 epochs.

Risk ranking. A bilinear score head trained with margin ranking loss on heuristic positives (negative-polarity infons and endpoints of `CONTRADICTS` edges) produces a scalar risk per infon. Aggregated to actor-level mean risk, the head correctly surfaces Honda (which has hedged claims and a contradiction edge) above actors with only positive statements. Training drives the margin loss to exactly zero on eight positives and forty-one negatives.

Anomaly localization. A bottleneck autoencoder over the frozen infon embeddings trains self-supervised with MSE reconstruction loss. At inference time, reconstruction error ranks infons by how far they sit from the corpus-typical pattern. No labels are required.

Root-cause analysis. A CausalView filters the hypergraph to its infon-to-infon CAUSES and NEXT edges, greedily accepting weight-sorted edges while rejecting anything that would create a cycle, and exposes ancestor, descendant, and path queries. The `root_cause(target)` method combines three signals: integrated- gradient attribution to the target's SUPPORTS mass, the self- supervised anomaly score of each ancestor, and the inverse shortest-path distance. On a Honda delays-EV target, the top four root causes are (in order) Honda producing solid-state batteries, Honda producing batteries, Honda investing in solid state, and Honda investing in batteries — each at distance 1 and with distinct anomaly scores. This is a DoWhy-style RCA view, computed in the learned embedding space rather than on a hand-drawn causal DAG.

Root-cause pipeline.

```

1. trained GNN                                -> h for every infon
2. refine()                                  -> CAUSES / CONTRADICTS / NEXT
                                           edges (from IF-THEN aggregator)
3. causal_view()                             -> DAG on CAUSES + NEXT
                                           (weight-greedy, cycle-free)
4. target infon:
   ancestors_with_distance()                 -> { ancestor : shortest distance }
5. attribute(target, "supports")             -> integrated-gradient score
                                           per input node
6. anomaly_head.score()                     -> reconstruction error per infon
7. combine:
   root_score = |IG| * anomaly * 1 / (1 + distance)

ranked ancestors    ->    top-k root causes with full provenance
                        (subject / predicate / object / path)

```

Counterfactual do-operator. A single `counterfactual()` method supports four intervention types: remove an infon and its edges, zero a node's features, drop a specific typed edge, or `do_anchor(src, replacement)` which replaces an anchor's feature vector with another's and propagates through the frozen GNN. On a Toyota-subject target, `do_anchor(toyota → catl)` moves the SUPPORTS mass from 0.787 to 0.841 and θ from 0.194 to 0.143 — the kind of textbook shift a Pearl do-operator should produce.

Refutation. A `refute(target, intervention, n_trials)` method compares the observed intervention effect against random-placebo interventions of the same structural type. The p-value-like statistic is the fraction of placebos whose $|\Delta|$ matches or exceeds the observed; a value near 1.0 means the observed effect is not uniquely attributable to the treatment — the signature of a confounded or high-shared-structure neighbourhood. On a deliberately confounded corpus (where a china anchor appears in the neighbourhood of both a catl treatment and a honda outcome) the test correctly flags `p_value_like = 0.875` on a subject-side anchor swap, because seven of eight random placebos reproduced a larger effect than the named intervention.

Refute: placebo permutation test.

```

target                                baseline_mass
|                                     ^
| counterfactual(intervention)       |
v                                     |
perturbed graph -> forward -> MassReadout -> observed |Delta|

for trial in 1..N:
    random placebo intervention         placebo_mass_i
    (same structural type,              ^
     random substitution)              |
    |                                  |
    v                                  |
    perturbed graph -> forward -> MassReadout -> placebo |Delta|_i

p_value_like = (# placebos with |Delta| >= observed) / N

```

```

p ~ 0  -> observed effect stands out: likely unique
p ~ 1  -> most placebos match: confounded / shared-structure

```

Recommender. Five interaction anchors (adopts, prefers, avoids, selects, rejects) turn naturally-occurring interaction sentences into (user, item, polarity) tuples. A bilinear scoring head trained with Bayesian Personalized Ranking on these tuples learns actor→feature preferences — in our corpus Toyota's top recommendation is ev (which its sentences explicitly "adopt"), and `explain_recommendation()` traces the prediction back to the underlying "Toyota adopts" infons via integrated gradients. A diversity regularizer on the top-5 items prevents concentration collapse. BPR loss drops from 37.4 to 0.004 over 45 epochs.

Subgraph and graph-level classification. A `SubgraphPool` module (mean, sum, max, or learned attention) pools node embeddings over any index set, letting every node-level head also operate at subgraph or whole-graph level. Pooling all 17 Toyota-subject infons and applying the mass readout gives a well-defined per-actor verdict with a single number for each of SUPPORTS, REFUTES, UNCERTAIN and θ .

New-infon prediction. Adding a new infon to the graph (a sentence about an unseen event involving existing anchors) requires only a forward pass: the new node attaches via its spoke edges, the trained message passing propagates into it, and any head can then read out on it. No retraining is necessary. This is the standard property of trained-then-transductive GNNs, and it transfers cleanly to all nine heads above.

End-to-end lifecycle.

```

documents
|
| extract_infons (SPLADE + schema projection)
v
infons + spoke edges + LOCATED_AT + NEXT (surface)
|
| HypergraphBuilder.build(schema)
v
HyperGraph (node features + 8 typed edges)
|
| fit(): DS teacher (polarity, alignment, anchor
| distance, confidence) combined by Dempster's
| rule + sheaf-coherence regularizer
v
trained GNN (backbone weights frozen for head training)
|
+-----> refine(): add CAUSES, CONTRADICTS,
| additional NEXT edges via IF-THEN
|
+-----> train_next_head / train_time_to_event_head /
| train_risk_head / train_anomaly_head /
| train_recommender_head
| (GNN frozen; only the head parameters move)
v
query-time:
reason() -> DS verdict
predict_next_anchors()-> IF-THEN top-k
predict_time_to_event -> Weibull params
rank_risk() -> ranked entities
score_anomalies() -> ranked infons
counterfactual() -> baseline vs interventional mass
refute() -> observed vs placebo deltas
attribute() -> per-node IG explanations
root_cause() -> IG x anomaly x 1/distance
recommend() -> bilinear-scored items
discover_anchors() -> spectral Kan extension

```

Discussion

The system we describe is empirical engineering more than theoretical advance. The underlying components are well established: the infon representation [Barwise 1983]; the IKRIS Knowledge Language [Hayes & Menzel 2006]; relational graph convolution [Schlichtkrull et al. 2018]; sparse retrieval [Formal et al. 2021]; Dempster–Shafer belief theory [Shafer 1976]; spectral clustering [Shi & Malik 2000]; Kan extensions [Mac Lane 1978]. Several authors have combined subsets of these ingredients. Neural sheaf diffusion [Bodnar et al. 2022] uses sheaf structure inside the propagation rule rather than as a regularizer, and evidential deep learning [Sensoy et al. 2018] outputs Dirichlet parameters interpretable as belief masses in a classification setting. Our contribution is a particular way of fitting the pieces together so that the propagation rule itself becomes an interpreter for a small compositional logic, while the output space preserves the distinction between disagreement and ignorance.

Three properties of the resulting system are worth emphasizing.

First, ignorance is preserved throughout the computation. The Θ mass is a scalar in $[0, 1]$ that is propagated, combined, and read out at every stage; it is not inferred post-hoc from prediction entropy. On queries whose evidence is genuinely thin, the Θ mass stays high after training — as in the Toyota and Honda queries above — and this provides a directly usable signal for downstream triage.

Second, the aggregators are interpretable. A forward pass through the network can be transcribed, one layer at a time, into a short logical expression over the graph. We do not claim a formal soundness result: the network computes a learned approximation whose inductive bias is compositional logic. But for diagnostic purposes — asking why a particular query got a particular mass — the trace is short enough to read.

Third, the schema is an explicit knob. A larger or better-curated schema produces cleaner extractions, and the anchor discovery step provides concrete suggestions for extending the schema when new structure appears in the corpus. This contrasts with implicit schema in embedding-based systems, where the "anchors" are distributed across the model weights and cannot be audited or edited.

A note on confounding. The refutation test is not a formal identifiability proof, but it is a practical diagnostic that surfaces the same failure mode a DoWhy-style backdoor check would catch. On a synthetic corpus designed to contain a shared-parent structure — a 'china' anchor that co-occurs with both a 'catl' treatment anchor and a 'honda' outcome anchor — running `refute()` on a `do_anchor(catl → tesla)` intervention produces a p-value-like statistic of 0.875, meaning that seven of eight random anchor substitutions reproduce a larger mass shift than the named intervention. This is the behavioural signature of confounding: the effect is not unique to the treatment, it is distributed across a shared neighbourhood. The test also exposes an architectural asymmetry we had not anticipated: because the three spoke edges (INITIATES, ASSERTS, TARGETS) are directed, only subject-anchor interventions propagate into the infon node. Object and predicate substitutions have no effect on the target's mass under two-layer propagation. A reader interested in full bidirectional causal propagation would either add reverse spoke edges or rely on the infon-to-infon NEXT and CAUSES edges discovered during refinement. We report the asymmetry transparently rather than hiding it, because it has implications for what kinds of causal claims the system can make honestly.

The system has real limitations. The teacher is heuristic and inherits the biases of its four sources; the coherence regularizer mitigates but does not fix this. Training is transductive and we have not demonstrated cross-graph transfer. The minimum-based AND is one reasonable choice of connective realization but not the only one; product or Łukasiewicz variants may behave differently on corpora where conjunction should be softer. The present evaluation is a controlled scenario designed for traceability rather than a benchmark. A companion paper addressing open-world claim verification at scale is in preparation.

A broader point. Contemporary AI systems tend to be evaluated on their average-case accuracy and their worst-case robustness; they are less often evaluated on whether they can, on demand, say "I don't know in

a principled way." The representation described here makes that capability structural rather than emergent, and the cost of the representation — a small schema, a sparse encoder, a two-layer graph network — is modest enough to run locally. Whether the approach scales to the domains where this capability matters most — scientific review, medical reasoning, legal analysis — depends on whether an adequate schema can be built for those domains. Our anchor discovery step is one attempt to make that construction tractable.

Methods

Corpus and schema. The evaluation corpus consists of five hand-authored documents describing a synthetic electric-vehicle-battery scenario (mean 9 sentences per document). The schema contains 22 anchors across four types: 5 actors (toyota, honda, tesla, panasonic, catl), 6 relations (invests, partners, produces, expands, delays, acquires), 5 features (battery, solid_state, ev, factory, supply_chain), 3 markets (japan, north_america, china), and 3 auxiliary entries. Each anchor definition lists surface tokens; actor and market anchors also carry country-code and macro-region metadata.

Encoder and projection. The encoder is a SPLADE-tiny model (vocab 30 522) with a random seeded projection onto 64 dimensions. The same seed is used throughout so that rebuilt features are stable across runs. Projected features are L2-normalized.

Extraction. For each sentence, the encoder produces a sparse activation vector. Anchors whose activations exceed threshold (0.2 by default) are candidates for role fillers. Valid triples (subject-actor, predicate-relation, object-feature-or-actor) are emitted with confidence equal to the minimum role activation. Tense is inferred from surface cues; polarity is inferred from explicit negation words; temporal references are parsed from date expressions.

Network. The network has two message-passing blocks of hidden size 64. Each block maintains eight W_r matrices (one per relation), a W_{self} matrix, and dedicated modules for the AND aggregator (gate-and-min), the OR aggregator (gate-and-max), the IF-THEN aggregator (attention gate), and the context gate. Layer normalization and ReLU follow the aggregation. The mass readout is a single linear layer from 64 dimensions to 4 logits followed by softmax.

Teacher construction. The teacher mass per infon is the Dempster combination of four components, each a belief mass function over $\{\text{SUPPORTS}, \text{REFUTES}, \text{UNCERTAIN}, \Theta\}$: (1) polarity, which biases toward SUPPORTS or REFUTES depending on surface polarity; (2) triple alignment, which rewards type matches in the (subject, predicate, object) slots; (3) anchor distance, which penalizes degenerate triples where subject and object overlap excessively; (4) confidence, which maps extraction confidence to SUPPORTS mass and low confidence to Θ .

Coherence score. We build the anchor co-occurrence matrix from sentence encodings thresholded at 0.3, compute NPMI on the pair counts, and form the graph Laplacian from the positive-NPMI entries. The Fiedler value is the second-smallest eigenvalue. An infon's coherence score is the mean pairwise NPMI of its three anchors, mapped affinely from $[-1, 1]$ to $[0, 1]$.

Training. Adam optimizer, $\text{lr} = 1\text{e-}3$, weight decay 0, batch mode (full graph per step), gradient clipping at norm 1.0. Maximum 30 epochs with patience 8 and improvement threshold $1\text{e-}5$, applied after a minimum of 10 epochs. The total loss is $\text{KL}(\text{predicted} \parallel \text{teacher}) + 0.2 \cdot \text{coherence_penalty}$, where coherence_penalty is the corpus-mean of $(1 - m(\Theta)) \cdot (1 - \text{coherence})$.

Refinement. Infons are grouped by subject anchor and ordered by tense using the map $\text{past} \rightarrow 0$, $\text{present} \rightarrow 1$, $\text{present-continuous} \rightarrow 1$, $\text{future} \rightarrow 2$, $\text{conditional} \rightarrow 2$, $\text{unknown} \rightarrow -1$. For each temporally ordered pair with distinct predicates, the premise and conclusion embeddings are stacked and pushed through the IF-THEN aggregator in a single batch. The resulting embeddings are read out by the mass readout; pairs with $m(\text{SUPPORTS}) > 0.6$ become CAUSES edges, pairs with $m(\text{REFUTES}) > 0.6$ become CONTRADICTS edges, and all temporally ordered pairs (regardless of predicate) become NEXT edges.

Anchor discovery. Anchor-node embeddings from the trained network are L2-normalized; pairwise cosine similarity produces a weighted affinity matrix (diagonal zeroed, negative entries clipped to 0). The normalized Laplacian is decomposed; the first k non-trivial eigenvectors form a spectral embedding on which k -means with k -means++ initialization produces cluster assignments. A discovered anchor is named by its highest-degree member, typed by its majority member type, and tokenized by concatenation of member surface tokens. The silhouette coefficient is computed in the original cosine-affinity space. The procedure is the concrete realization of the left Kan extension described in Results; no explicit category-theoretic machinery is constructed.

Reproducibility. All experiments use random seed 42. The implementation is released as the `cognition` package under Apache-2.0, with both a local SQLite backend and an AWS backend (DynamoDB + S3 + Lambda). Package weights total 17 MB (SPLADE- tiny in safetensors format). A full test suite including the eight scenarios reported here runs in under 25 seconds on a single CPU.

Data availability. The synthetic corpus and schema used for the reported results are included in the package as unit tests (tests/test_logic.py). No external datasets were used in the reported evaluation.

Code availability. The cognition package and all experimental scripts will be released with the final manuscript.

Figure Captions

Figure 1. Per-relation update rules in the infon hypergraph. Each edge type carries a learned weight matrix W_r and an aggregation rule chosen to match the edge's meaning: an AND-style minimum for the three spokes of a triple (top); a directional IF-THEN attention for temporal and logical edges (middle); a context-gated product for spatial edges (bottom). Message passing composes these operations, two layers deep.

Figure 2. (a) Input versus layer-2 feature norms for the scenario graph: non-trivial aggregation produces consistent separation. (b) Training loss curves with and without coherence regularization; both converge, but the regularized loss allocates more residual mass to Θ for structurally weak triples. (c) The anchor co-occurrence graph with positive-NPMI edges; the Fiedler value summarizes global connectivity. (d) Example compound-query evaluations showing belief-mass outputs for nested "for-all", "there-exists", AND, and negation expressions.

References

[abbreviated; full bibliography to follow]

Barwise & Perry 1983, Situations and Attitudes (MIT Press).

Shafer 1976, A Mathematical Theory of Evidence (Princeton University Press).

Hayes & Menzel 2006, IKL as the basis for a unified logical framework, Proc. IKRIS Workshop. Mac Lane 1978, Categories for the Working Mathematician (Springer). Schlichtkrull et al. 2018, Modeling relational data with graph convolutional networks, Proc. ESWC. Formal et al. 2021, SPLADE: sparse lexical and expansion model for first stage ranking, Proc. SIGIR. Sensoy et al. 2018, Evidential deep learning to quantify classification uncertainty, NeurIPS. Bodnar et al. 2022, Neural sheaf diffusion, NeurIPS. Jousselme et al. 2001, A new distance between two bodies of evidence, Information Fusion.

Shi & Malik 2000, Normalized cuts and image segmentation, IEEE TPAMI.